

Lab Manual — Week 2

Concurrency & Distributed Systems — Lebanese University, Faculty of Engineering

Topic: Threads and Lifecycle

Learning Objectives

By the end of Week 2, you will understand:

1. **Thread Creation:** How to spawn threads and when they truly start executing.
2. **Thread Lifecycle:** States (NEW, RUNNABLE, WAITING, TIMED_WAITING, TERMINATED).
3. **Join and Futures:** How to wait for thread completion.
4. **Thread Interleaving:** Why you cannot assume order or fairness.
5. **Coordination Primitives:** `Object.wait/notify`, `CountDownLatch`, `Semaphore`.
6. **Race Conditions:** Basic races and the need for synchronization.
7. **Sleep vs Synchronization:** Why `Thread.sleep` cannot replace proper coordination.

Lab Structure

Week 2 is organized into 3 parts (day 1), each with 5–10 demos and 1–2 exercises:

Part	Topic	Demos + Exercises
Part 1	Basics (creation, start, sleep)	Demo01–Demo06 + Ex1
Part 2	Lifecycle (join, lifecycle states)	Demo07–Demo15 + Ex2
Part 3	Coordination (wait/notify, latches)	Demo16–Demo22 + Ex3

1 Part 1: Basics — Thread Creation and Sleep

1.1 Demo01: First Thread

Goal: Create and start a thread; observe that it runs concurrently.

Logic:

- Subclass `Thread` or implement `Runnable`.
- Override `run()` method with work.
- Call `thread.start()`, not `run()`.
- `start()` creates OS thread; `run()` calls it directly (no concurrency).

Code Pattern:

```
Thread t = new Thread() -> {  
    System.out.println("Hello from thread");  
};  
t.start(); // Spawns OS thread  
t.run(); // WRONG: calls directly, no concurrency
```

Key Insight: Always call `start()`, never `run()`. Calling `run()` defeats concurrency.

1.2 Demo02: Two Threads, Interleaving

Goal: Show that two threads execute concurrently and interleave.

Logic:

- Create Thread A and Thread B.
- Both print messages in a loop.
- Output shows interleaved messages, not sequential.
- Demonstrates scheduler control.

Key Insight: Threads are not sequential. You don't control when they run. Interleaving is unpredictable.

1.3 Demo03: The Sleep Myth

Goal: Show why `Thread.sleep()` is not a synchronization primitive.

Logic:

- Thread A sleeps 100ms, assuming Thread B finishes by then.
- Thread B might be slow (GC, cache miss, scheduler delay).
- Thread A wakes up before Thread B finishes.
- Result: race condition, not safety.

Key Insight: `sleep()` is a suggestion to the scheduler, not a contract. It cannot replace real coordination.

1.4 Demo04: Thread Naming

Goal: Name threads for debugging.

Logic:

- `thread.setName("worker-1")`.
- `Thread.currentThread().getName()`.
- Helpful in logs and thread dumps.

Key Insight: Thread names are free debugging aids. Use them!

1.5 Demo05: Is Alive and Active

Goal: Query thread state.

Logic:

- `thread.isAlive()`: true if thread hasn't terminated.
- Can be `false` before `start()`.
- Becomes `false` after `run()` completes.

Key Insight: A thread is “alive” from `start()` until `run()` returns.

1.6 Demo06: Daemon Threads

Goal: Understand daemon vs non-daemon threads.

Logic:

- Daemon threads: JVM exits even if daemons are still running.
- Non-daemon threads: JVM waits for them to complete.
- Use for background work (logging, cleanup).

Code Pattern:

```
Thread daemon = new Thread(task);
daemon.setDaemon(true);
daemon.start();
```

2 Part 2: Lifecycle — Join and State Transitions

2.1 Demo07: Join

Goal: Wait for a thread to complete.

Logic:

- Spawn Thread A.
- Call `thread.join()` from main thread.
- Main blocks until Thread A terminates.
- Useful for sequential coordination.

Code Pattern:

```
Thread t = new Thread(task);
t.start();
t.join(); // Blocks until t finishes
System.out.println("t finished");
```

Key Insight: `join()` is the basic “wait for thread” operation. Use it when you need to sequence tasks.

2.2 Demo08–Demo15: States, Priorities, Scheduling

Goal: Understand thread states and scheduling behavior.

Logic:

- NEW: created, not started.

- **RUNNABLE**: eligible to run, may or may not have CPU.
- **WAITING**: blocked on `Object.wait()`, `join()`, etc.
- **TIMED_WAITING**: blocked with timeout.
- **TERMINATED**: `run()` finished.

Demo09: Priorities

- `thread.setPriority(Thread.MAX_PRIORITY)` (1–10 scale).
- Hints to scheduler; not guaranteed order.
- Use sparingly; can cause starvation.

Key Insight: Thread states are transitions, not guarantees. The scheduler has the final say.

2.3 Demo14: Sleep Does Not Release Locks

Goal: Show that `sleep()` holds locks acquired by `synchronized`.

Logic:

- Thread A enters `synchronized` method, then sleeps.
- Thread B tries to enter same `synchronized` method.
- Thread B blocks (even though Thread A is sleeping).
- After Thread A wakes, Thread B can proceed.

Key Insight: `sleep()` does not release locks. Use `Object.wait()` for that.

3 Part 3: Coordination — Wait/Notify and Latches

3.1 Demo16–Demo21: Wait/Notify

Goal: Use `Object.wait()` and `notify()` for coordination.

Logic:

- Thread A calls `synchronized(lock) lock.wait();` .
- Thread A blocks, releases the lock, and waits for notification.
- Thread B acquires lock, does work, calls `lock.notifyAll()`.
- Thread A wakes, reacquires lock, continues.

Code Pattern:

```
synchronized(lock) {
    while (!condition) {
        lock.wait(); // Release lock, wait
    }
    // Proceed
}

synchronized(lock) {
    condition = true;
    lock.notifyAll(); // Wake waiters
}
```

Key Insight: `wait()` releases the lock; other threads can proceed. `notify()` wakes one waiter.

3.2 Demo22: Semaphore and CountdownLatch

Goal: Higher-level coordination primitives.

Logic:

- `CountDownLatch(N)`: N threads signal when done; main waits.
- `Semaphore(N)`: N permits. `acquire()` waits if none available. `release()` adds permit.
- Cleaner than raw `wait/notify`.

Code Pattern:

```
CountDownLatch latch = new CountDownLatch(3);  
// In threads: latch.countDown();  
// In main: latch.await();  
  
Semaphore sem = new Semaphore(2);  
sem.acquire(); // Block if no permits  
// Use resource  
sem.release(); // Release permit
```

Key Insight: Latches and semaphores are safe, composable coordination tools. Prefer them over raw `wait/notify`.

Summary: The Thread Model

Week 2 teaches **threads as concurrent execution units**.

1. **Creation:** Call `start()`, not `run()`.
2. **Interleaving:** You don't control order; expect nondeterminism.
3. **Lifecycle:** Threads transition through states.
4. **Join:** Wait for thread completion.
5. **Coordination:** Use `wait/notify`, latches, semaphores.
6. **Sleep is not sync:** Never use `sleep` for coordination.

All together: **concurrent threads that communicate safely**.